

Reviewer Pack — Minimal Index of Monodromy Representation Bounds Communicati...

Entry fa01f0bfba2b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-31 05:58:15 UTC

Minimal Index of Monodromy Representation Bounds Communication Complexity

Entry ID: fa01f0bfba2b **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-31 05:58:15 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (specifically, monodromy representations)

Field B (complexity object): Communication Complexity

Statement:

For every boolean function $f: \{0,1\}^n \rightarrow \{0,1\}$, the communication complexity of f is upper-bounded by the minimal index of a monodromy representation associated with its associated elliptic curve, i.e., $C(f) \leq \min I(e_f)$, where $C(f)$ denotes the communication complexity of f and e_f represents the elliptic curve associated with f .

Rationale (proposer's reasoning):

The minimal index of a monodromy representation can capture intricate arithmetic properties that are not evident in the original boolean function. These properties might reveal hidden structures in the communication process, leading to lower bounds on communication complexity. The bridge between algebraic geometry and communication complexity may expose new approaches for proving hardness results.

Taxonomy category: monodromy_representation (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): b885e167e0884409

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all boolean functions $f: \{0,1\}^n \rightarrow \{0,1\}$ with $n \leq 40$, the communication complexity $C(f)$ is less than or equal to the minimal index of a monodromy representation $I(e_f)$ associated with its elliptic curve, and this condition holds true across at least 95% of all seeds tested.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "monodromy representation" AND "communication complexity" - "elliptic curve" AND communication complexity AND algebraic geometry" - algebraic geometry monodromy AND communication complexity upper bound

Top relevant hits considered: - [http://arxiv.org/abs/1712.01659v1] Shifted Poisson geometry and meromorphic matrix algebras over an elliptic curve - [http://arxiv.org/abs/2407.00692v1] The motivic fundamental group of a punctured elliptic curve and algebraic cycles - [http://arxiv.org/abs/math/0508553v1] On the Hall algebra of an elliptic curve, II - [http://arxiv.org/abs/1912.00347v3] Equiresidual algebraic geometry I: The affine theory - [http://arxiv.org/abs/2204.10334v1] Machine Learning Algebraic Geometry for Physics - [http://arxiv.org/abs/1409.1534v1] Algorithms in Real Algebraic Geometry: A Survey - [s2:10.1080/17513057.2023.2274559] Subaltern perspectives of developing communication campaigns: Re-examining the culture-centered approach in addressing h - [s2:c33660f74ad2cb933fb6b8c972e58593877e7af8] Making a Case for Spatial Prompting in Human-Robot Communication

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=240.5s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def compute_associated_elliptic_curve(f):
        # Placeholder function to simulate computation of an elliptic curve
        # This is a dummy implementation and should be replaced with actual logic
        return random.randint(1, 100)

    def minimal_index_of_monodromy_representation(e_f):
        # Placeholder function to simulate computation of the minimal index
        # This is a dummy implementation and should be replaced with actual logic
        return random.randint(1, 50)
```

```

communication_complexity = lambda f: len(f) / 2

results = []
for n in [5, 10, 15, 20, 30, 40]:
    instances_tested = 0
    n_max = n
    conjecture_holds = True
    counterexample = ""

    for _ in range(50):
        f = generate_boolean_function(n)
        e_f = compute_associated_elliptic_curve(f)
        I_e_f = minimal_index_of_monodromy_representation(e_f)
        C_f = communication_complexity(f)

        if C_f > I_e_f:
            conjecture_holds = False
            counterexample = f"n={n}, f={f}, e_f={e_f}, I(e_f)={I_e_f}, C(f)={C_f}"
            break

        instances_tested += 1

    results.append({
        "metric_name": "communication_complexity",
        "metric_value": C_f,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    })

mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value) ** 2 for result in results) / len(results))
support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

return {
    "seed": seed,
    "mean_metric_value": mean_metric_value,
    "std_metric_value": std_metric_value,
    "support_fraction": support_fraction
}

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [random.randint(2, 1000) for _ in range(30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric_value = sum(result["mean_metric_value"] for result in results) / len(results)
    std_metric_value = math.sqrt(sum((result["mean_metric_value"] - mean_metric_value) ** 2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["support_fraction"] >= 0.95) / len(results)

```

```

if support_fraction >= 0.95:
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
elif any(not result["conjecture_holds"] for result in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_seed={first_failing_seed}")
else:
    print(f"RESULT: INCONCLUSIVE reason=insufficient_support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means we cannot verify the conjecture for all boolean functions $f: \{0,1\}^n \rightarrow \{0,1\}$ with $n \leq 40$ as required by the pre-registered support condition. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15130
2	preregistration	ollama_remote	glm4:latest	0	0	17216
3	novelty	ollama_remote	glm4:latest	0	0	21083
4	novelty	ollama_remote	glm4:latest	0	0	10940
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15526
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	53452
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8525
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	33527
9	judge	ollama_remote	glm4:latest	0	0	53583

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 228982 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in [research/audit/fa01f0bfba2b.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/fa01f0bfba2b.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/fa01f0bfba2b.tar.gz` (if generated)